

# Dashboard Hydra — Projeto Águas Vivas (IFSC)

Sistema avançado de telemetria e monitoramento ambiental desenvolvido pela equipe de **Engenharia Eletrônica** do Instituto Federal de Santa Catarina (IFSC).

Este arquivo contém o guia específico para a branch de produção utilizando **Podman**, que adota uma arquitetura híbrida de deploy.

---

## 1. Arquitetura de Produção Híbrida (IFSC)

Para garantir máxima performance, disponibilidade e isolamento de segurança: - **Frontend (Interface do Usuário)**: Hospedado e distribuído globalmente através do **Cloudflare Pages**. Ele se comunica diretamente com a API do backend de produção. - **Backend & Banco de Dados (Stack de Dados)**: Rodando em containers utilizando **Podman** em uma Máquina Virtual no servidor interno do IFSC. - **Broker MQTT**: Conectado à nuvem da Shiftr.io para coleta de telemetria em tempo real.

---

## 2. Deploy e Gerenciamento com Podman

O **Podman** (Pod Manager) é utilizado em substituição ao Docker por questões de conformidade de segurança e execução rootless no servidor da instituição.

### 2.1. Como Subir a Stack pela Primeira Vez

1. Acesse o servidor Linux da VM do IFSC.
2. Certifique-se de estar na branch de deploy:  

```
git checkout feat/deploy-hibrido-ifsc
```
3. Suba os containers (Postgres, API Backend e o servidor web do Frontend):  

```
podman-compose up -d --build
```

### 2.2. Comandos Úteis do Podman para Operação e Manutenção

Como o Podman gerencia containers de forma semelhante ao Docker, os comandos de depuração são muito parecidos:

- **Verificar o status dos containers:** `bash podman ps -a`
- **Visualizar logs em tempo real (essencial para debugar erros na API ou conexões MQTT):** `bash podman logs -f hydra_backend`
- **Acompanhar logs do Banco de Dados:** `bash podman logs -f hydra_postgres`
- **Parar a execução de toda a stack:** `bash podman-compose down`

- **Reiniciar um serviço específico (ex: reiniciar backend após ajuste de config):**  
`bash podman restart hydra_backend`
  - **Verificar consumo de recursos (CPU/Memória) dos containers:** `bash podman stats`
- 

### 3. Acesso Administrativo e Criação do Primeiro Administrador

Ao rodar a stack em um banco de dados novo, as tabelas do PostgreSQL são criadas zeradas pelo Prisma ORM (`npx prisma migrate deploy`), sem registros na tabela `Usuario`. Para habilitar o primeiro login na interface administrativa, você deve cadastrar o administrador padrão.

- **E-mail padrão a ser criado:** `admin@hydra.local`
- **Senha padrão a ser criada:** `admin123`

*Como rodar o script de criação do administrador utilizando o Podman:*

Com a stack de containers ativa no servidor, execute o comando abaixo direto do terminal do hospedeiro:

```
podman exec -it hydra_backend npx tsx src/createAdmin.ts
```

*(Alternativamente, se preferir usar o podman-compose no diretório do projeto: `podman-compose exec backend npx tsx src/createAdmin.ts`)*

Após a execução com sucesso, acesse a área administrativa do sistema (em ambiente de deploy híbrido ou local) e realize o login com as credenciais padrão. **Recomenda-se alterar a senha imediatamente na aba “Gerenciar Usuários”.**

---

### 4. Ingestão de Dados via MQTT (Tempo Real)

As boias e gateways enviam dados telemétricos em tempo real publicando mensagens JSON no broker MQTT configurado.

- **Tópico de publicação:** `Hydra/<boia_id>` (onde `<boia_id>` é o identificador único da boia cadastrada no painel, em letras minúsculas, ex: `boia_01`).
- **Formato do payload (JSON):**

```
json {      "timestamp": "2026-06-02 16:25",
    "lat": -27.593708,
    "lon": -48.542835,
    "alt": 16.6,
    "tempAr": 25.3,
    "umidAr": 70,
    "pressao": 1012,
    "indiceUV": 5,
    "chuvaAcum": 0,
    "ventoVel": 12,
    "ventoDir": 180,
    "tempAgua": 22.1,
    "phAgua": 7.2,
    "condutivEC": 980,
    "turbidez": 12
  }
```

[!NOTE] Os parâmetros de geolocalização (`lat`, `lon`, `alt`) são opcionais. Caso a boia seja estática, a localização informada no painel administrativo será mantida. Todos os campos numéricos de sensores são opcionais, permitindo enviar payloads reduzidos se necessário.

---

## 5. Ingestão de Dados via CSV (Lote)

Para cadastrar grandes volumes de dados históricos salvos em cartões SD ou registros passados, utilize a importação de arquivos CSV na interface de gerenciamento de boias (acessada pelo usuário admin).

- **Separador:** Ponto e vírgula (;).
- **Codificação recomendada:** UTF-8.
- **Cabeçalho obrigatório (Primeira Linha):** csv  
timestamp;tempAr;umidAr;pressao;indiceUV;chuvaAcum;ventoVel;ventoDir;tempAgua;phAgua;condutivEC;turbidez

### Exemplo de formato aceito (dados\_historicos.csv):

```
timestamp;tempAr;umidAr;pressao;indiceUV;chuvaAcum;ventoVel;ventoDir;t  
empAgua;phAgua;condutivEC;turbidez  
2026-05-01 10:00;25.3;70;1012;5;0;12;180;22.1;7.2;980;12  
2026-05-01 10:15;25.1;72;1011.8;4;0;11;175;22.0;7.1;978;13  
2026-05-01 10:30;24.9;75;1011.5;4;0.2;14;190;21.9;7.2;982;12
```

[!IMPORTANT] O formato do campo timestamp deve seguir o padrão YYYY-MM-DD HH:MM (ex: 2026-05-01 10:00) ou o padrão ISO 8601. Para valores com decimais nos sensores, utilize o caractere de ponto (.) como separador decimal.

---

## 6. Configuração de Ambiente de Produção

Arquivos-chave de configuração na branch feat/deploy-hibrido-ifsc: -  
.env.production: Configuração de variáveis do frontend para build de produção (aponta para a URL externa da API do backend). - backend/Containerfile &  
Containerfile.frontend: Arquivos de build para os containers no Podman. - podman-compose.yml: Arquivo de orquestração local de serviços da stack. -  
backend/prisma/schema.prisma: Definição de tabelas do banco de dados relacional.

---